

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Практическая работа 5-6 и Лабораторная работа 2-4

Выполнил:

Смирнова Карина

К3341

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

ЛР2: Реализация микросервисов

Задание:

Опираясь на документ, сформированный в рамках Д34, реализуйте разделение вашего бэкенд-приложения на микросервисы.

Д35: Реализация межсервисного взаимодействия посредством очередей сообщений

ЛР3: Контейнеризация написанного приложения средствами docker

Задание:

- реализовать Dockerfile для каждого сервиса;
- написать общий docker-compose.yml;
- настроить сетевое взаимодействие между сервисами.

ЛР4: Развёртывание приложения на удалённом сервере

Д36: Настройка Gitlab CI/Github actions для автоматического развёртывания бэкенд-приложения

Задание:

Необходимо настроить автодеплой (с триггером на обновление кода в вашем репозитории, на определённой ветке) для вашего приложения на удалённый сервер с использованием Github Actions или Gitlab CI (любая другая CI-система также может быть использована).

Ход работы

Renting Service - проект на микро сервисной архитектуре + Kafka + Docker

Этот репозиторий - Spring Boot backend на Java 21 для системы аренды недвижимости. Проект находится в переходном состоянии: по коду это

один Maven-модуль и один jar, но в Docker он запускается как набор микросервисов, потому что один и тот же jar стартует несколько раз с разными Spring-профилями (`SPRING_PROFILES_ACTIVE`).

1) Состав системы в Docker

В `compose.yaml` поднимаются контейнеры:

- gateway (nginx) - единая точка входа: `http://localhost:8080`
- user-service - регистрация/авторизация/профиль
- property-service - объявления, аренда и платежи
- communication-service - чаты и сообщения
- notification-service - обработка событий и отправка уведомлений по email
- postgres - один контейнер Postgres (PostGIS), но с несколькими отдельными БД
- kafka - брокер сообщений для асинхронных событий
- mailhog - SMTP песочница для писем + веб-интерфейс

2) Nginx gateway

Клиент общается только с gateway:

Клиент → `http://localhost:8080` → (nginx) → нужный сервис

Маршрутизация делается по URL-префиксу в `docker/nginx/nginx.conf`:

- `/api/v1/auth/**` , `/api/v1/users/**` → user-service `/api/v1/listings/**` ,
- `/api/v1/rents/**` , `/api/v1/bookings/**` , `/api/v1/payments/**` → property-service
- `/api/v1/chats/**` , `/api/v1/internal/chats` → communication-service

Также nginx проксирует Swagger UI по сервисам:

- `/docs/user/...` → Swagger user-service
- `/docs/property/...` → Swagger
- property-service `/docs/communication/...` → Swagger communication-service

3) Внешние и внутренние запросы

В системе есть внешние (public) пути - для клиента через gateway

Это обычные ручки, которыми пользуется пользователь:

/api/v1/auth/... - регистрация/логин/refresh/logout

/api/v1/users/... - профиль, повторная отправка письма подтверждения

/api/v1/listings/... - объявления

/api/v1/rents/... - заявки на долгосрочную аренду

/api/v1/chats/... - чаты

Есть внутренние (internal) пути - для межсервисного общения

Это ручки сервис → сервис, чтобы не тащить чужие таблицы к себе и не ломать границы:

- user-service:
 - GET /api/v1/internal/users/{userId} - отдать данные пользователя другим сервисам
 - POST /api/v1/auth/validate - проверить JWT и вернуть userId
- property-service:
 - GET /api/v1/listings/internal/listings/{listingId} - отдать данные объявления другим сервисам
- communication-service:
 - POST /api/v1/internal/chats — создать чат по запросу другого сервиса

4) Базы данных

Поднимается один контейнер Postgres, но в нем создаются 4 отдельные базы (инициализация в docker/postgres/init/01-create-databases.sql):

- user_db
- property_db
- comm_db
- notification_db

Каждый сервис подключается к своей базе через env-переменные (DB_NAME=...) в compose.yaml.

Смысл этого разделения:

- у каждого сервиса свои данные (ownership)
- проще мигрировать к настоящим микросервисам дальше
- меньше риск случайно залезть в чужие таблицы

5) Использование Kafka

Kafka здесь - это шина событий (асинхронный канал), чтобы сервисы не дергали друг друга напрямую для уведомлений.

Логика:

- доменный сервис делает действие (например, создана заявка на аренду)
- публикует событие в Kafka
- notification-service подписан на эти события и отправляет уведомление

В проекте:

- NotificationEventPublisher публикует JSON в топик notification-events
- NotificationEventConsumer слушает notification-events (groupId notification-service) и отправляет email

Схема:

property-service

→ Kafka (topic: notification-events)

→ notification-service

→ SMTP (mailhog:1025)

6) Использование MailHog

MailHog - это SMTP эмулятор и web-интерфейс для просмотра писем. Он нужен, чтобы:

- не отправлять письма в реальную почту во время разработки
- легко проверить письмо пришло/не пришло, и что внутри него

Порты: SMTP:

- mailhog:1025 (внутри Docker)
- Web UI: <http://localhost:8025> (в браузере)

В проекте:

- при регистрации отправляется письмо подтверждения email
- при создании записи на долгосрочную аренду владельцу объявления приходит уведомление
- при создании записи на краткосрочную аренду владельцу объявления приходит уведомление
- после оплаты краткосрочной аренды владельцу объявления приходит уведомление

7) Пример цепочки вызовов: от регистрации до заявки на долгосрочную аренду

Пример реального пользовательского сценария.

Шаг 1: Регистрация пользователя (user-service через gateway)

Запрос:

- POST <http://localhost:8080/api/v1/auth/register>

Пример тела:

```
{
  "email": "user1@mail.com",
  "username": "user1",
  "phone": "+79990000000",
  "password": "secret123"
}
```

Что происходит:

- nginx отправляет запрос в user-service
- пользователь сохраняется в user_db
- отправляется письмо подтверждения на SMTP mailhog:1025
- письмо видно в MailHog UI: <http://localhost:8025>

Шаг 2: Подтверждение email

Берем токен из письма (MailHog) и подтверждаем:

- POST <http://localhost:8080/api/v1/users/email/confirm>

```
{ "token": "..." }
```

Шаг 3: Логин и получение JWT

- POST <http://localhost:8080/api/v1/auth/login>

```
{ "email": "user1@mail.com", "password": "secret123" }
```

Ответ содержит accessToken (Bearer) и refreshToken .

Шаг 4: Выбор MONTHLY-объявления (property-service)

- GET <http://localhost:8080/api/v1/listings?rentMode=MONTHLY...>
- GET <http://localhost:8080/api/v1/listings/{listingId}>

Шаг 5: Создание заявки на долгосрочную аренду (property-service)

Запрос (с Bearer accessToken):

- POST <http://localhost:8080/api/v1/rents>

```
{"listingId": 123, "communicationMethod": "CHAT" }
```

Что происходит внутри:

- Проверка подлинности пользователя (аутентификация):
 - gateway прокидывает заголовок Authorization: Bearer ... в property-service
 - в property-service JWT-фильтр (Spring Security) проверяет подпись токена и достает userId , который дальше

используется в бизнес-логике (например, через `SecurityUtils.currentUserId()`)

- `property-service` создает заявку в `property_db`
- если выбран ЧАТ , то `property-service` делает внутренний вызов:
 - `POST http://communication-service:8080/api/v1/internal/chats`
- затем `property-service` публикует событие уведомления в Kafka (топик `notification events`)
- `notification-service` читает событие и отправляет email через MailHog

Итоговая схема:

Клиент

→ `gateway (/api/v1/rents)`
→ `property-service`
→ (HTTP) `communication-service (/api/v1/internal/chats)` [если ЧАТ]
→ (Kafka) `notification-events`
→ `notification-service`
→ MailHog SMTP

8) Сборка и запуск

Запустить можно командой:

`docker compose up --build`

Точки входа:

- Gateway: `http://localhost:8080`
- MailHog UI: `http://localhost:8025`
- Kafka: `localhost:9092`
- Postgres: `localhost:5433`

9) Деплой на сервер

Для деплоя используется тот же `compose.yaml` . Код хранится в GitHub, а на VPS лежит только рабочая копия репозитория и `.env` с секретами.

Как это работает:

- при `push` в ветку `server` запускается GitHub Actions

- workflow подключается к VPS по SSH
- переходит в папку VPS_DEPLOY_PATH
- делает git pull --ff-only origin server
- пересобирает образы и перезапускает контейнеры
- дополнительно перезапускает gateway , чтобы Nginx подхватил новые контейнеры

На сервере после деплоя поднимаются:

- gateway на :8080
- mailhog на :8085
- user-service , property-service , communication-service , notification-service
- postgres и kafka

Для работы деплоя нужны GitHub Secrets:

- VPS_HOST
- VPS_PORT
- VPS_USER
- VPS_SSH_KEY
- VPS_DEPLOY_PATH

На сервере отдельно лежит файл .env с переменными:

- POSTGRES_PASSWORD
- JWT_SECRET
- PAYMENT_COMMISSION_PERCENT
- PAYMENT MOCK_SUCCESS_RATE

10) GitHub Actions

Workflow находится в .github/workflows/deploy-vps.yml и запускается при push в server . Это значит, что любая новая версия кода в этой ветке автоматически уходит на сервер.

Порядок внутри workflow такой:

- GitHub Actions получает доступ к VPS по SSH
- заходит в папку с проектом
- обновляет код из ветки server

- выполняет `docker compose up -d --build`
- делает `docker compose restart gateway`
- показывает состояние контейнеров
- удаляет старые образы командой `docker image prune -f`

Таким образом при изменении кода в ветке `server` и `push` в GitHub происходит автоматическое обновление и перезапуск проекта на удаленном сервере.

Документация доступна по пути:

- property: <http://157.22.230.55:8080/docs/property/swagger-ui.html>
- user: <http://157.22.230.55:8080/docs/user/swagger-ui/index.html>
- communication:
<http://157.22.230.55:8080/docs/communication/swagger-ui.html>

Вывод

В ходе работы была собрана и настроена микросервисная топология системы аренды в Docker:

- настроен `nginx gateway` для маршрутизации запросов и удобного доступа к Swagger по префиксам `/docs/*`
- приложение запускается как набор сервисов (`user-service` , `property-service` , `communication-service` , `notification-service`) за счет Spring-профилей
- данные разделены по отдельным базам (`user_db` , `property_db` , `comm_db` , `notification_db`) внутри одного контейнера Postgres
- добавлен Kafka для асинхронной отправки сообщений на почту
- добавлен MailHog, чтобы безопасно проверять отправку писем (подтверждение email, уведомления) без реальной почты
- Приложение развернуто на удаленном сервере
- Настроен CI/CD для обновления приложения при изменении ветки `server`